3.3.3 Using the linear function with gradient descent

We proceed with the linear mean function, which is in the form of $\mu = \mathbf{w}_T\mathbf{x} + b$, where μ is the predictive mean value at the test point $\mathbf{X}$, while $\mathbf{w}$ is the weight vector that concatenates the coefficients for each of the features in $\mathbf{X}$, and b is a constant bias term.

By using the linear mean function, we are encoding the assumption that the expected behavior of our objective function is equal to a linear combination of the features of the data point $\mathbf{X}$. For our housing price example, we only have one feature, living area, and we expect it to have a positive weight, so that by increasing the living area, our model will predict a higher price.

Another way to think about this linear-mean model is that we have a linear regression model (which also assumes the target label to be a linear combination of the features) and we put a probabilistic belief that is a GP model on top of our predictions. This gives us the power offered by a linear regression model, while maintaining all the benefits of modeling with a GP, namely uncertainty quantification.

> **NOTE** **Linear mean versus constant mean**
>
> **Under a constant mean function, the weight vector w is fixed at the zero vector, and the bias b is the mean value that we learned to optimize in the previous subsection. In other words, the linear function is a more general model than the constant mean function.**

Implementation-wise, building a GP model with a linear mean function is quite straightforward: we simply swap out our constant mean and replace it with a `gpytorch.means.LinearMean` instance as follows (our `forward()` method remains unchanged):

```
class LinearMeanGPModel(gpytorch.models.ExactGP):
    def __init__(self, train_x, train_y, likelihood):
        super().__init__(train_x, train_y, likelihood)
        self.mean_module = gpytorch.means.LinearMean(1)   ❶
        self.covar_module = gpytorch.kernels.RBFKernel()
```

❶ Linear mean.

Here, we use `1` to initialize our mean module to indicate that we are working with a one-dimensional objective function. When working with a higher-dimensional function, you can simply pass in the dimensionality of that function here. Aside from this, everything else about our model is similar to what we had before.

Fitting and training this new model on our three-point data set, we obtain the predictions in Figure 3.8.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>